



Universidad Autónoma de Nuevo León
Facultad de Ingenieria Mecanica y Electrica

Informa de Arquitectura

Unidad 3 - Diseño arquitectural del sistema con IA

FerreMax

Materia: Componentes de Lógica de Negocios
Gpo:003 M6
Equipo 17

Integrantes:
Cristian Aleli Garcia Avalos
Abdul Ali Vazquez Flores 2227013
Juan Manuel Casanova Avila 2269091

San Nicolás de los Garza. NL. 17 de Octubre del 2026

Índice

1. Alcance..... 1
Análisis Operativo (AS-IS y Problemas Centrales)..... 1
Etapa 1.....2
2. Comparación y Selección del Patrón de Lógica de Negocio..... 3
3. Mapa de Dominio y Responsabilidades (DDD)..... 6
Lenguaje Ubicuo..... 6
Imagen 1 Mapa de Dominio Ferremax..... 7
Entidades y Value Objects.....8
Fichas de Aggregates..... 9
Aggregate 1: Cotización.....9
Aggregate 2: Venta..... 9
Aggregate 3: Cliente Empresarial..... 10
Aggregate 4: Existencia / Inventario..... 10
Aggregate 5: Transferencia.....10
Aggregate 6: Pedido..... 10
Aggregate 7: Hoja de Ruta.....11
Aggregate 8: Devolución..... 11
4. Diseño de la Service Layer (Etapa 4)..... 12
5. Separación de Persistencia y Transacciones (Etapa 5).....13
6. Separación de Commands y Queries (etapa 6)..... 15
7. Revisión de propuesta con SOLID (Etapa 7).....16
1. Principio de Responsabilidad Única (SRP - Single Responsibility Principle)... 16
2. Principio de Abierto / Cerrado (OCP - Open / Closed Principle)..... 16
3. Principio de Sustitución de Liskov (LSP - Liskov Substitution Principle).....17
4. Principio de Segregación de Interfaces (ISP - Interface Segregation Principle).. 17
5. Principio de Inversión de Dependencias (DIP - Dependency Inversion Principle)..... 18

8. Architecture Decision Records (ADRs)..... 19

Fichas Formales de Decisiones Arquitecturales..... 19

ADR-001: Selección del Patrón de Lógica de Negocio (Domain Model).....19

ADR-002: Desacoplamiento de Persistencia mediante Repository Pattern y Gestión Transaccional con Unit of Work.....20

ADR-003: Aplicación de CQRS Básico para la Separación de Escritura (Commands) y Lectura (Queries)..... 21

Diagramas UML.....23

Diagrama de clase..... 23

Diagrama de Secuencia CU-01.....24

Diagrama de Secuencia CU-04.....25

9. Conclusiones..... 26

Impacto Arquitectural..... 26

Matriz de Trazabilidad..... 27

Bitácora de IA (entregable 5)..... 33

Código Base..... 35

1. Alcance

La situación operativa actual de FerreMax exhibe severos cuellos de botella derivados del uso de herramientas desconectadas, procedimientos manuales y la ausencia de una fuente única de verdad para el inventario y las condiciones crediticias. La gestión de cotizaciones mediante hojas de cálculo sin validación automática de vigencia, sumada a la incapacidad de consultar el stock consolidado en tiempo real entre las tres sucursales y el almacén central, impacta directamente los márgenes financieros por fluctuaciones de precios de proveedores y genera erosión en la confianza del cliente. Resolver estas deficiencias operativas mediante una arquitectura desacoplada resulta crítico para estabilizar la operación diaria y asegurar la rentabilidad del negocio.

Análisis Operativo (AS-IS y Problemas Centrales)

- **Venta de mercancía ya comprometida (doble venta)**: Falta de actualización e integración de stock en tiempo real entre las sucursales y el almacén central. Ocurre cuando dos vendedores en distintos puntos consultan o venden el mismo producto disponible de manera concurrente.
- **Cotizaciones en hojas de cálculo**: Ausencia de un bloqueo automático por fecha de vencimiento, lo que permite la aplicación indebida de cotizaciones vencidas en caja, absorbiendo incrementos de costos de proveedores y erosionando el margen de ganancia.
- **Visibilidad limitada del inventario**: Inexistencia de un catálogo centralizado que despliegue de forma inmediata existencias locales, stock global, niveles mínimos de reabastecimiento y ubicaciones físicas (pasillos y anaqueles) para los vendedores.
- **Validación manual de crédito**: Dependencia de llamadas telefónicas al departamento de cobranza para validar saldos y líneas de crédito de clientes empresariales, lo que congestiona el flujo de cobro y retrasa la atención en sucursal.
- **Reportes gerenciales manuales:** Elaboración extenuante de informes consolidados mediante hojas de cálculo en Excel, lo que demanda horas de trabajo administrativo y demora la toma de decisiones estratégicas.

Etapa 1

ID	Caso de uso	Actor	Reglas relacionadas	Resultado esperado
CU-01	Venta directa en mostrador con producto disponible en sucursal	Vendedor / Cliente	Consultar existencia, mínimo y ubicación del SKU en la sucursal. Verificar que la mercancía no esté comprometida. Aplicar descuento automático según tipo de cliente (5% regular). Reservar y descontar el stock en tiempo real al concretar la venta.	Se genera el ticket y se descuenta el stock en tiempo real. Si el producto llega a su mínimo, se notifica a Compras.
CU-02	Autorización de una venta a crédito para un cliente empresarial	Caja/Crédito (solicita el Vendedor)	Revisar condiciones comerciales y disponibilidad autorizada. Validar en tiempo real la línea de crédito y las facturas vencidas con el código del cliente. Si excede el límite o tiene saldo vencido: bloquear y mostrar “Crédito suspendido”. Solo un Gerente puede autorizar la excepción de crédito.	Venta autorizada: se registra a crédito, se aplica el descuento (10% empresarial) y se descuenta el stock. Si no procede, se rechaza la venta (o queda sujeta a la excepción del Gerente).
CU-03	Surtir un pedido mediante transferencia porque no hay stock en la sucursal actual	Vendedor / Almacén	Alertar al vendedor cuando no hay stock local y ofrecer la transferencia. Mostrar existencia por sucursal y almacén. Marcar la mercancía transferida como “en tránsito”. Bloquear su venta hasta confirmar la recepción.	Se genera la transferencia y la mercancía queda “en tránsito”. Al confirmar la recepción, se libera el stock y se surte el pedido.
CU-04	Intentar aplicar una cotización que ya está vencida	Vendedor / Cliente	Verificar la vigencia de la cotización: 5 días cliente regular; 3 días empresarial o mayoreo. Bloquear automáticamente las cotizaciones vencidas. Solo el Gerente puede autorizar una excepción.	Se rechaza la aplicación de la cotización y se informa que está vencida. Se requiere una nueva cotización con precios vigentes o la autorización del Gerente.

2. Comparación y Selección del Patrón de Lógica de Negocio

Se analizarán tres patrones de arquitecturas (Transaction Script, Table Module y Domain Model) basándonos en la complejidad de las reglas de negocio de FerreMax documentadas en la unidad 2, los cambios esperados y el manejo de los estados de la empresa.

A continuación, se presenta la tabla comparativa:

Criterio del caso	Transaction Script	Table Module	Domain Module	Evidencia en U2
Complejidad de reglas	Agrupará todas las reglas de descuentos (5% o 10%) y vigencias (3 o 5 días) en un solo método procedimental gigante, volviéndolo difícil de mantener	Separaría la lógica por tablas de base de datos (ej. Cotizacion, Cliente), pero la validación de negocio seguiría ligada a los datos crudos y no al comportamiento.	Permite crear objetos ricos (ej. Cotizacion, ClienteEmpresarial) que encapsulan sus propias reglas para calcular descuentos y fechas de vencimiento de forma natural.	BR005, BR006, US-005, US 006.
Cambios esperados	Si cambian las políticas de entrega a domicilio (zonas de riesgo, horarios), habría que rastrear y modificar múltiples scripts dispersos que procesen ventas.	Facilitaría agregar columnas a la tabla de envíos, pero complicaría agregar nuevas reglas de negocio condicionales que involucren múltiples tablas a la vez.	Las políticas de entrega se pueden modelar como estrategias o servicios de dominio aislados, permitiendo escalar las reglas sin afectar el resto del sistema.	BR011, US-011.
Interacción entre conceptos	Una venta a crédito requeriría un script que llame a inventario, luego a cobranza y luego a rutas, generando un alto acoplamiento	La lógica de una venta parcial obligaría a que el módulo de "Ventas" tenga que conocer e interactuar íntimamente con la estructura de las tablas de "Inventario" y "Backorder".	Un Aggregate de Orden puede orquestar la interacción fluida con Inventario y LineaDeCredito, validando que todo se cumpla antes de persistir los cambios.	BR007, BR012, US-007, US 012.

Estados e invariantes	El control del estado de la mercancía ("en tránsito", "reservado") dependería de que el programador no olvide actualizar un campo en la base de datos en cada script.	Manejaría los cambios de estado actualizando registros de una tabla, pero no protegería las transacciones si otro módulo altera la base directamente.	Protege las invariantes de forma estricta: un objeto Cliente no permitirá procesar una venta si su estado interno es "Crédito suspendido", garantizando consistencia.	BR008, BR013, US-008, US 013.
Criterio adicional: Concurrencia y reservas en tiempo real	Bloquearía tablas completas o registros sueltos de manera ineficiente durante la reserva de stock, afectando a las 3 sucursales.	Dependería fuertemente de los bloqueos de base de datos a nivel de tabla para evitar la doble venta, limitando la escalabilidad.	El modelo de dominio gestiona la concurrencia a nivel de Aggregate Root (ej. Producto), asegurando que la reserva en tiempo real sea atómica y segura.	BR001, US-001, US-019

La argumentación de la decisión que se tomó:
Para el caso de FerreMax, se descarta Transaction Script porque el sistema no es un simple CRUD. Las operaciones principales desencadenan múltiples validaciones y efectos secundarios. Por ejemplo, concretar una venta requiere descontar stock en tiempo real (BR001), validar líneas de crédito y saldos vencidos (BR007), y potencialmente generar un backorder si el pedido es parcial (BR012). Implementar esto como scripts transaccionales resultaría en código duplicado y propenso a errores, especialmente cuando el negocio escale.

Table Module también se descarta porque, aunque organiza mejor el código alrededor de las tablas de la base de datos, FerreMax requiere proteger invariantes estrictas que cruzan múltiples conceptos. Por ejemplo, bloquear ventas cuando el límite de crédito se excede (BR008) o asegurar que las cotizaciones vencidas no se apliquen sin la excepción de un gerente (BR004, BR009). El modelo tabular no maneja bien esta lógica comportamental compleja.

La elección de **Domain Model** es la más adecuada para FerreMax por las siguientes razones respaldadas en el análisis de requerimientos:

- **Protección de Invariantes Complejas:** FerreMax dicta que una mercancía transferida debe marcarse como "en tránsito" y bloquearse para la venta hasta que la otra sucursal confirme su recepción (BR013). Un modelo de dominio permite modelar la Transferencia y el Inventario como entidades con estados claros que impiden alteraciones indebidas.
- **Encapsulamiento de Lógica Condicional:** La asignación de la vigencia de cotizaciones (5 días regular, 3 días empresarial) (BR005) y los descuentos automáticos (5% y 10%) (BR006) dejarán de ser sentencias if/else dispersas en controladores. Estas reglas vivirán dentro de la entidad Cotizacion, calculándose intrínsecamente según el tipo de cliente.
- **Transiciones Seguras y Autorizaciones:** Requerimientos como la excepción para ventas sin stock (BR016) o la autorización de excepciones de crédito (BR009) requieren un manejo de estados rico. El modelo de dominio permite que entidades como Venta solo avancen de estado si se inyecta un objeto de AutorizacionGerencial.

Implementar el Modelo de Dominio exige una curva de aprendizaje mayor para el equipo, ya que requiere separar estrictamente el código de infraestructura (bases de datos, UI) del núcleo del negocio. Nos obligará a mapear objetos de memoria a tablas relacionales utilizando un ORM o repositorios personalizados en las siguientes etapas, lo cual añade una capa extra de complejidad técnica en comparación con las consultas SQL directas. Sin embargo, este costo se justifica plenamente para erradicar el problema crítico actual del negocio: las ventas de mercancía ya comprometida (BR019).

3. Mapa de Dominio y Responsabilidades (DDD)

El diseño guiado por el dominio (DDD) permite traducir el lenguaje operativo y las reglas de negocio de FerreMax en estructuras de software aisladas de la tecnología de persistencia o interfaz gráfica. Este enfoque asegura que el código exprese con absoluta fidelidad las intenciones de la operación diaria.

Lenguaje Ubicuo

El lenguaje ubicuo unifica la terminología utilizada por los involucrados en el negocio y el equipo de arquitectura:

- **Mercancía comprometida:** Producto ya vendido o reservado en una transacción que no debe ofrecerse ni venderse a otro cliente.
- **Cartera vencida:** Saldo acumulado por clientes empresariales que han superado el plazo límite de pago establecido sin haber liquidado sus facturas.
- Crédito suspendido: Estado del cliente empresarial que inhabilita automáticamente el procesamiento de nuevas compras a crédito por exceso de límite o facturas vencidas.
- **Backorder (orden pendiente):** Registro automático de partidas faltantes en un pedido parcial, el cual queda pendiente para su reabastecimiento y entrega posterior.
- **Hoja de ruta:** Documento logístico que organiza y agrupa las entregas a domicilio asignadas a un transporte basándose en códigos postales.
- **Modo offline:** Capacidad del sistema en sucursal para procesar transacciones locales ante pérdidas temporales de conexión a Internet y sincronizarlas posteriormente con el servidor central.
- **Vigencia de cotización:** Periodo de validez temporal en días (5 días regular, 3 días empresarial) durante el cual se garantizan los precios de una cotización.
- **SKU (Stock Keeping Unit)**: Código alfa-numérico único que identifica de forma unívoca a cada producto dentro del catálogo.
- **Existencia:** Cantidad física de unidades de un SKU disponibles en una ubicación específica (sucursal o almacén central).

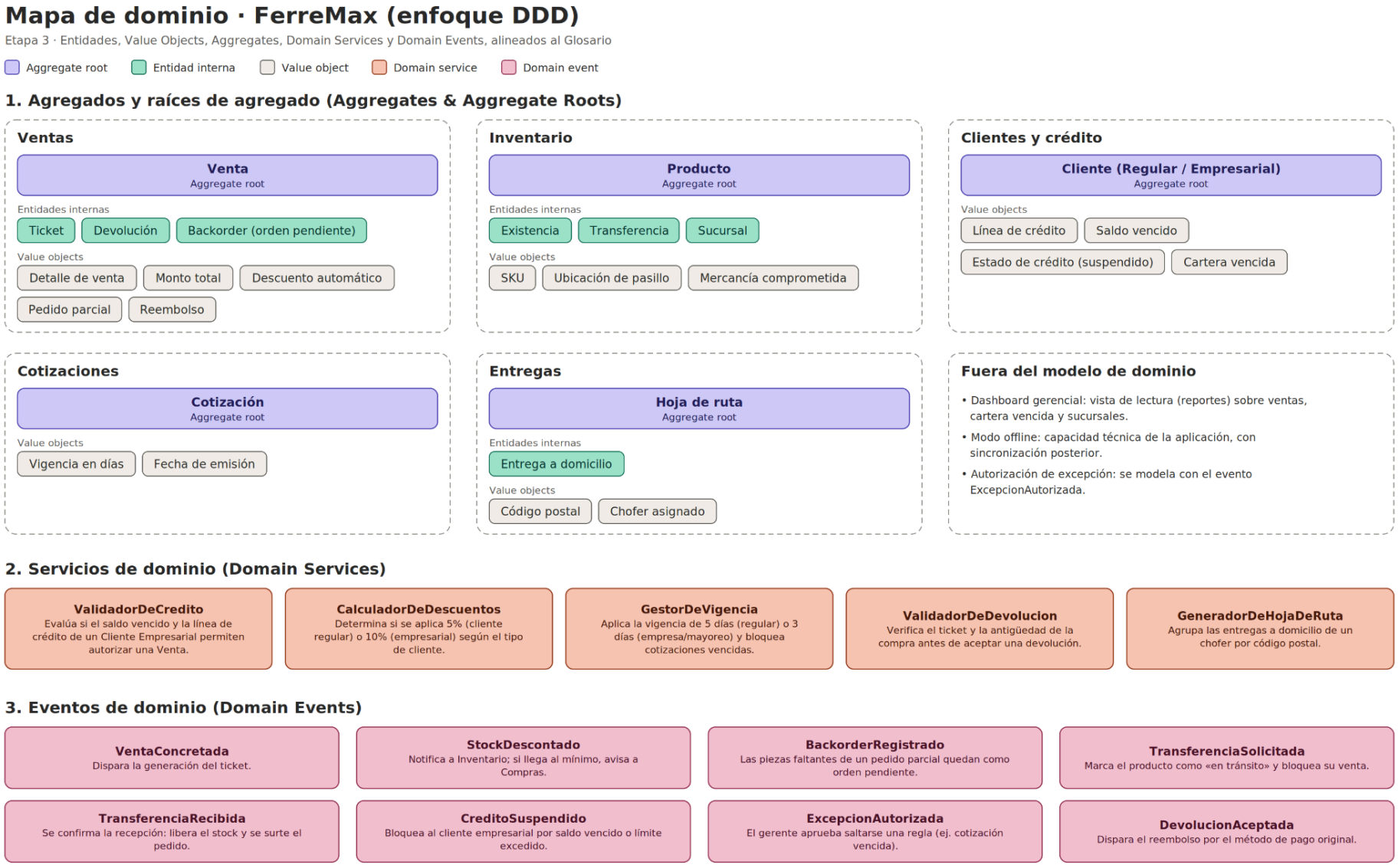


Imagen 1 Mapa de Dominio Ferremax

Entidades y Value Objects

El modelo distingue claramente los objetos que poseen una identidad continua a lo largo del tiempo de aquellos definidos únicamente por el valor de sus atributos:

- **Entidades** (con identidad y ciclo de vida):
    - **Cliente:** Representa al sujeto que realiza compras; identificado por codigoCliente. Posee métodos del modelo UML como solicitarCotizacion() y comprar().
    - **ClienteEmpresarial:** Especialización de Cliente con ciclo de vida crediticio, lineaCredito, saldoVencido y estadoCredito. Expone el método validarCredito().
    - **Cotizacion:** Documento comercial con vigencia; identificado por folio. Posee los atributos fechaEmision, vigenciaDias, estado y los métodos estaVigente() y aplicarDescuento().
    - **Venta:** Registro atómico de la transacción comercial; identificada por folio. Posee fecha, tipoPago, descuento, total y los métodos calcularTotal() y generarTicket().
    - **Producto:** Artículo del catálogo; identificado por sku, con descripcion, categoria, unidad y precioCatalogo.
    - **Existencia:** Control de volumen físico de un SKU en una sucursal; posee cantidad, minimo, ubicacionPasillo y los métodos descontar(cantidad) y llegoAlMinimo().
    - **Sucursal:** Punto de venta o almacenamiento físico; posee nombre, tipo y el método consultarExistencia(sku).
    - **Transferencia:** Movimiento logístico entre sucursales; posee fecha, cantidad, estado y los métodos marcarEnTransito() y confirmarRecepcion().
    - **Ticket:** Comprobante físico/digital emitido; posee numero, fecha y el método imprimir().
    - **DetalleVenta:** Partida individual que vincula Producto con Venta, registrando cantidad, precioUnitario y expone calcularSubtotal().

- **Value Objects** (inmutables, definidos por su valor)**:**
    - **PoliticaVigencia**: Define las reglas y cálculo de días hábiles de vigencia de las cotizaciones.
    - **PoliticaDescuento**: Establece los porcentajes automáticos de descuento (5% o 10%).
    - **EstadoMercancia**: Define las transiciones de disponibilidad ("Disponible", "Comprometida", "En tránsito").
    - **AntiguedadTicket**: Almacena y evalúa el cálculo de días transcurridos para validar si procede una devolución (máximo 30 días).
    - **MetodoPago**: Define la vía original del flujo monetario (Efectivo, Tarjeta, Crédito).
    - **Backorder:** Modela las cantidades faltantes pendientes de surtir en un pedido parcial.

Fichas de Aggregates

El Mapa de Dominio (Figura 1)agrupa visualmente los conceptos en 5 grandes bloques/raíces de consistencia principal (Ventas, Inventario, Clientes y Crédito, Cotizaciones, Entregas), los cuales se detallan individualmente en las 8 fichas de agregados a continuación para un desglose exhaustivo de sus invariantes y responsabilidades.

Aggregate 1: Cotización

- **Aggregate Root:** `Cotizacion`
- **Responsabilidades:** Asignar la vigencia temporal según el tipo de cliente, calcular precios finales aplicando descuentos automáticos (`aplicarDescuento()`) y evaluar si el folio continúa activo (`estaVigente()`).
- **Invariantes Protegidas:** Una cotización vencida (`estaVigente() == false`) no puede pasar a estado "Aplicada" sin recibir la credencial autorizada (PIN) de un Gerente. Los descuentos automáticos (5% regular, 10% empresarial) no pueden modificarse de forma arbitraria.

Aggregate 2: Venta

- **Aggregate Root:** `Venta`
- **Responsabilidades:** Orquestar el cálculo del importe total de la compra (`calcularTotal()`), generar las partidas de `DetalleVenta` con `calcularSubtotal()`, asociar el comprobante `Ticket` (`generarTicket()`) y validar los métodos de pago.
- **Invariantes Protegidas:** El monto `total` debe coincidir exactamente con la suma de los subtotales de sus partidas menos el `descuento` aplicado. Una

venta a crédito no puede consolidarse si el estado crediticio del cliente no es válido.

Aggregate 3: Cliente Empresarial

- **Aggregate Root:** ClienteEmpresarial
- **Responsabilidades:** Administrar el límite autorizado de crédito (lineaCredito), acumular el saldo pendiente (saldoVencido) y ejecutar la validación crediticia (validarCredito()).
- **Invariantes Protegidas:** El saldo adeudado más el monto de la compra actual no debe exceder la lineaCredito. Si existen facturas con saldoVencido mayor a cero, el método validarCredito() pasa el estadoCredito a "Crédito suspendido", bloqueando transacciones automáticas a crédito

Aggregate 4: Existencia / Inventario

- **Aggregate Root:** Existencia
- **Responsabilidades:** Registrar la disponibilidad física por SKU y sucursal, descontar stock ante confirmación de venta (descontar(cantidad)) y verificar el umbral de reabastecimiento (llegoAlMinimo()).
- **Invariantes Protegidas:** La cantidad en stock no puede disminuir por debajo de cero en operaciones normales. Ante reducciones que alcancen o superen el umbral minimo, la entidad activa la notificación del límite.

Aggregate 5: Transferencia

- **Aggregate Root:** Transferencia
- **Responsabilidades:** Administrar la movilización de productos entre un origen y un destino, cambiando estados mediante marcarEnTransito() y confirmarRecepcion().
- **Invariantes Protegidas:** Al ejecutar marcarEnTransito(), la mercancía vinculada debe adquirir el estado "en tránsito", quedando bloqueada para la venta en ambas sucursales hasta que la sucursal de destino ejecute confirmarRecepcion().

Aggregate 6: Pedido

- **Aggregate Root:** Pedido
- **Responsabilidades:** Gestionar la consolidación de compras con entregas programadas o surtidos diferidos.
- **Invariantes Protegidas:** Ante falta de stock para completar un pedido, solo se factura la cantidad disponible de inmediato y el remanente genera un objeto Backorder automático.

Aggregate 7: Hoja de Ruta

- **Aggregate Root:** HojaDeRuta
- **Responsabilidades:** Consolidar envíos a domicilio y agrupar entregas por cercanía geográfica (Código Postal).
- **Invariantes Protegidas:** No se pueden añadir entregas a una hoja de ruta si sobrepasan la capacidad del vehículo, si la colonia pertenece a una zona marcada como insegura o si la hora de programación supera las 6:00 PM.

Aggregate 8: Devolución

- **Aggregate Root:** Devolucion
- **Responsabilidades:** Procesar el retorno de productos y la restitución de dinero.
- **Invariantes Protegidas:** Solo se autoriza si la fecha del Ticket presenta una antigüedad menor o igual a 30 días y el reembolso debe ejecutarse obligatoriamente mediante el mismo MetodoPago de la venta original.

4. Diseño de la Service Layer (Etapa 4)

En esta etapa se diseñan los Application Services encargados de exponer y coordinar los casos de uso principales del sistema FerreMax (Ventas, Créditos, Inventario y Cotizaciones). La justificación de este diseño radica en mantener la capa de servicios como un orquestador puro. Su única responsabilidad es recibir la solicitud del usuario, obtener los objetos necesarios mediante los repositorios y coordinar la transacción, delegando todas las reglas de negocio complejas directamente a las entidades del modelo de dominio. Esto evita que la lógica de negocio se acople a la aplicación y cumple estrictamente con el patrón Domain Model.

Servicio	Método o Caso de uso	Entrada	Colaboradores (repositorio)	Salida	Reglas que delega al Dominio
VentaAppService	ProcesarVentaDirecta (CU-01)	CodigoCliente, Lista<SKU, Cantidad>IdSucursal	ClienteRepository, InventarioRepository, VentARepository, UnitOfWork	Ticket o Mensaje de Error	Validación de existencia, descuento del 5% regular y reserva de stock en tiempo real (delegado a entidades Cliente y Existencia).
CreditoAppService	AutorizarVentaCredito (CU-02)	CodigoCliente, TotalVenta	ClienteRepository, VentaRepository, UnitOfWork	VentaAutorizada o CréditoSuspendido	Validación de línea de crédito, saldo vencido y aplicación del descuento del 10% empresarial (delegado a ClienteEmpresarial).
InventarioAppService	SolicitarTransferencia (CU-03)	SKU, SucursalOrigen, SucursalDestino, Cantidad	InventarioRepository ,TransferenciaRepository, UnitOfWork	EstadoTransferencia	Marcar mercancía "en tránsito" y bloquear su venta hasta confirmación de recepción (delegado a Transferencia).
CotizacionAppService	AplicarCotizacion (CU-04)	FolioCotizacion,CodigoCliente	CotizacionRepository, UnitOfWork	CotizacionAprobada o Rechazo	Validación de vigencia (5 días regular, 3 días empresarial) y bloqueo automático de cotizaciones vencidas (delegado a Cotizacion).

5. Separación de Persistencia y Transacciones (Etapa 5)

Para garantizar que nuestro modelo de dominio (Domain Model) se mantenga agnóstico a la infraestructura y bases de datos, se aplicó el patrón Repository. Las interfaces definidas a continuación hablan un lenguaje puro de negocio, ocultando cualquier detalle de consultas SQL o frameworks de persistencia. Estas interfaces interactúan directamente con las entidades principales (Aggregates) definidas en el Diagrama de Clases de la Etapa 3.

Diseño de Repositorios:

Repository	Necesidad del dominio	Métodos propuestos	Objeto que recupera o guarda
IClienteRepository	Gestionar datos, líneas de crédito y validación de saldos de los clientes.	ObtenerPorCodigo(codigo), ActualizarSaldoCliente(cliente)	Cliente, ClienteEmpresarial.
IInventarioRepository	Controlar niveles de stock, validar mínimos y gestionar traslados de mercancía.	ObtenerExistencia(sku, sucursal), ActualizarStock(existencia)	Existencia, Transferencia.
IVentaRepository	Registrar transaccione s exitosas, sus partidas y el comprobante generado.	GuardarVenta(venta), ObtenerPorFolio(folio)	Venta, DetalleVenta, Ticket.
ICotizacionRepository	Administrar folios, fechas de emisión y estados de cotizaciones.	ObtenerPorFolio(folio), GuardarCotizacion(cotizacion)	Cotizacion.

Definición del Alcance Transaccional (Unit of Work) Para asegurar la consistencia del sistema, implementamos el patrón Unit of Work.

- Inició: La transacción inicia en la capa de aplicación en el momento exacto en que se invoca el caso de uso.
- Agrupación de cambios: El Unit of Work rastrea en memoria todas las modificaciones coordinadas. Por ejemplo, al concretar una Venta, agrupa la creación de la venta, la deducción en la Existencia y la actualización del ClienteEmpresarial.

- Manejo de errores: Si el dominio lanza una excepción por romper una invariante, el Unit of Work aborta la operación y no hace el commit a la base de datos, garantizado que el inventario y el saldo se mantenga intactos.

6. Separación de Commands y Queries (etapa 6)

Aplicando el principio de segregación de responsabilidades, dividimos las operaciones del sistema utilizando un enfoque básico de CQRS. Esta decisión técnica permite que las operaciones que modifican el estado del negocio estén fuertemente protegidas por el Modelo de Dominio, mientras que las necesidades de información pueden realizar lecturas rápidas sin instancias lógicas complejas, optimizando el rendimiento de la base de datos y evitando cuellos de botella en la aplicación.

Diseño de Operaciones:

Tipo	Nombre	Datos de entrada	Resultado (View Model/DTO)	Handler	Justificación
Command	RegistrarVentaCredito	CodigoCliente, Carritos<SKU, cantidad>, IdSucursal	ID de Venta	RegistrarVentaHandler	Operación de escritura que muta el estado del sistema: reduce el stock de la entidad Existencia y aumenta la deuda en ClienteEmpresarial (Aplica para CU-02).
Command	CrearTransferencia	SKU, Cantidad, SucursalOrigen, SucursalDestino	ID de Transferencia	TransferenciaHandler	Modifica el estado de la mercancía cambiándola a "en tránsito", lo cual bloquea temporalmente su venta (Aplica para CU-03).
Query	ConsultarStockYDetalles	SKU, IdSucursal	ExistenciaDTO (Stock, mínimo, pasillo)	ConsultarStockHandle	Satisface el CU 01. Es de solo lectura, no altera datos; devuelve información plana a la pantalla del vendedor sin pasar por lógicas complejas del dominio.
Query	VerificaVigenciaCotizacion	FolioCotizacion	CotizacionView (Estado, Días vigentes)	ConsultarCotizacionHandler	Permite leer el estatus actual de una cotización para validar rápidamente si sigue activa sin causar efectos secundarios en la base de datos.

7. Revisión de propuesta con SOLID (Etapa 7)

1. Principio de Responsabilidad Única (SRP - *Single Responsibility Principle*)

- **Principio Evaluado:** S — Cada clase o componente debe tener una única razón para cambiar.

- **Evidencia Encontrada:** En el diseño inicial de la capa de aplicación, el servicio VentaService asumió la responsabilidad de calcular los descuentos automáticos (5% regular y 10% empresarial), evaluar el límite de crédito del cliente y ejecutar la sentencia de persistencia en la base de datos.

- **Impacto / Riesgo:** Violación de SRP. El servicio acumulaba múltiples motivos de cambio independientemente de la orquestación: modificaciones en las reglas comerciales de descuento, cambios en las políticas financieras de cobro/crédito o actualizaciones en la tecnología de base de datos.

- **Cambio Realizado / Decisión Confirmada**: Se refactorizó la responsabilidad del cálculo de descuento delegando directamente al Agregado Cotizacion / Venta, la evaluación del crédito al Agregado ClienteEmpresarial y la gestión de la base de datos a las interfaces IVentaRepository y IUnitOfWork. El VentaService quedó acotado exclusivamente a coordinar el flujo del caso de uso sin absorber reglas de negocio

2. Principio de Abierto / Cerrado (OCP - *Open / Closed Principle*)

- **Principio Evaluado:** O — Las entidades de software deben estar abiertas a la extensión, pero cerradas a la modificación.

- **Evidencia Encontrada:** La lógica para calcular la fecha de vencimiento de cotizaciones (5 días para cliente regular y 3 días para cliente empresarial o mayoreo) dependía de una estructura condicional if-else o switch que evaluaba cadenas de texto fijas con el tipo de cliente.

- **Impacto / Riesgo:** Violación de OCP. Si FerreMax introduce un nuevo perfil comercial en el futuro (ej. "Distribuidor VIP" o "Gobierno") con reglas de vigencia o descuentos personalizados, se tendría que modificar el código interno de la clase Cotizacion, incrementando el riesgo de introducir errores en código probado.

- **Cambio Realizado / Decisión Confirmada**: Se abstrajo la regla de vigencias y descuentos mediante Value Objects polimórficos de políticas de cliente (PoliticaVigencia, PoliticaDescuento). De este modo, la incorporación de nuevos perfiles de cliente se realiza extendiendo e implementando nuevas políticas sin editar la clase Cotizacion existente.

3. Principio de Sustitución de Liskov (LSP - Liskov Substitution Principle)

- **Principio Evaluado:** L — Las clases derivadas deben poder sustituir a sus clases base sin alterar el comportamiento del sistema

- **Evidencia Encontrada:** Al diseñar la jerarquía entre la clase base Cliente y la clase derivada ClienteEmpresarial, la clase base incluía el método validarLineaCredito(). Sin embargo, al ejecutar este método sobre un ClienteParticular (venta de mostrador de contado), la clase arrojaba una excepción no controlada (NotSupportedException) porque los clientes particulares no manejan crédito ni saldo vencido

- **Impacto / Riesgo:** Violación de LSP. Sustituir un Cliente genérico por un ClienteParticular alteraba el contrato esperado e interrumpía el flujo de cobro inmediato en caja.

- **Cambio Realizado / Decisión Confirmada**: Se ajustó la jerarquía eliminando los métodos de crédito de la clase base Cliente. La clase Cliente únicamente conserva atributos globales de identificación, mientras que las reglas de línea de crédito, facturas vencidas y el estado de "Crédito suspendido" quedaron confinadas de forma segura dentro del Agregado ClienteEmpresarial.

4. Principio de Segregación de Interfaces (ISP - Interface Segregation Principle)

- **Principio Evaluado:** I — Ningún componente debe verse obligado a depender de métodos que no utiliza.

- **Evidencia Encontrada:** Se propuso inicialmente una interfaz de repositorio monolítica llamada IFerreMaxRepository que agrupaba todas las operaciones de consulta y almacenamiento del sistema (guardarVenta(), consultarStockSKU(), autorizarExcepcionCredito(), generarRutaLogistica()).

- **Impacto / Riesgo:** Violación de ISP. Componentes especializados como RuteoLogisticoService o CreditoService terminaban dependiendo de una interfaz gigante con decenas de métodos irrelevantes para su función operativa.

- **Cambio Realizado / Decisión Confirmada**: Se dividió la interfaz masiva en repositorios específicos enfocados en cada Agregado y contexto de negocio: IVentaRepository, IInventarioRepository, IClienteRepository e IHojaDeRutaRepository. Cada servicio de aplicación únicamente consume la interfaz acotada que requiere.

5. Principio de Inversión de Dependencias (DIP - Dependency Inversion Principle)

- **Principio Evaluado:** D — Los módulos de alto nivel no deben depender de módulos de bajo nivel; ambos deben depender de abstracciones.

- **Evidencia Encontrada:** En las primeras iteraciones, los servicios de la capa de Aplicación (CotizacionService, ReservaStockService) instanciaban directamente clases de infraestructura concretas (ej. MySQLVentaRepository o SqlServerDbContext).

- **Impacto / Riesgo:** Violación de DIP. La capa de Aplicación y las reglas del Dominio quedaban fuertemente acopladas al motor de base de datos relacional, impidiendo realizar pruebas unitarias independientes y dificultando la implementación del modo offline en sucursal.

- **Cambio Realizado / Decisión Confirmada**: Se aplicó inversión de dependencias mediante la inyección de abstracciones. El Dominio y la Aplicación dependen exclusivamente de la interfaz IVentaRepository y de la abstracción IUnitOfWork, mientras que las implementaciones tecnológicas de base de datos residen en la capa de Infrastructure y se inyectan en tiempo de ejecución.

8. Architecture Decision Records (ADRs)

Los Registros de Decisiones Arquitecturales (ADR) documentan el contexto, las alternativas evaluadas y las consecuencias de las elecciones estructurales realizadas en FerreMax. Preservan la memoria técnica del proyecto y justifican la solución adoptada.

Fichas Formales de Decisiones Arquitecturales

ADR-001: Selección del Patrón de Lógica de Negocio (Domain Model)

- **Estado:** Aceptada

- **Contexto:** FerreMax opera con tres sucursales y un almacén central atendiendo ventas de mostrador y clientes empresariales con línea de crédito. Se identificaron problemas operativos críticos como la venta de mercancía ya comprometida por falta de sincronización en tiempo real, el cobro de cotizaciones vencidas debido a controles en hojas de cálculo y retrasos en la validación de crédito. El sistema requiere proteger reglas e invariantes complejas (cálculo automático de descuentos del 5% y 10%, asignación estricta de vigencias de 3 y 5 días según el tipo de cliente, y bloqueo automático por crédito suspendido o facturas vencidas)

- **Alternativas:**
    1. **Transaction Script:** Organizar la lógica en procedimientos secuenciales por transacción. Consecuencia*:* Duplicación masiva de reglas de validación de crédito y cálculos de descuento en múltiples scripts o controladores de UI, lo que incrementa el riesgo de errores en ventas concurrentes y dificultará el mantenimiento

    2. **Table Module:** Organizar la lógica de negocio alrededor de tablas de la base de datos. Consecuencia*:* Dificultad para modelar relaciones y agregados complejos de negocio como Cotizacion, Venta y ClienteEmpresarial, acoplando fuertemente el diseño al esquema relacional de la base de datos.

    3. **Domain Model:** Modelo orientado a objetos donde entidades y agregados encapsulan sus propias reglas e invariantes de negocio. *Consecuencia:* Exige un mayor esfuerzo inicial de modelado, pero garantiza el aislamiento total de las reglas de negocio en el dominio y evita estados inconsistentes.

- **Decisión:** Adoptar el patrón **Domain Model** como patrón principal de lógica de negocio. Las entidades y agregados (Cotizacion, ClienteEmpresarial, InventarioLocal) gestionarán y protegerán sus propias reglas e invariantes sin depender de la infraestructura o la interfaz de usuario.

- **Consecuencias:**
    1. **Beneficios**: Encapsulamiento efectivo de las reglas de negocio clave como lo son los descuentos automáticos, vigencias y control de crédito, también una alta reusabilidad de componentes de dominio y facilidad para ejecutar pruebas unitarias aisladas de la base de datos.

    2. **Costos y Riesgos**: Curva de aprendizaje más elevada para el equipo y necesidad de capas adicionales de mapeo entre el modelo de dominio y la capa de persistencia.

- **Trazabilidad: BRD**: N1, N4, N5, N6, N7, N8; **Historias de Usuario**: US-001, US-004, US-005, US-006, US-007, US-008; **Glosario**: Cotización, Cliente empresarial, Crédito suspendido.

ADR-002: Desacoplamiento de Persistencia mediante Repository Pattern y Gestión Transaccional con Unit of Work

- **Estado:** Aceptada

- **Contexto:** Las operaciones del proceso principal de FerreMax involucran modificaciones atómicas distribuidas: descontar stock de sucursal o almacén central en tiempo real, marcar mercancía en transferencia como "en tránsito", registrar pedidos parciales con orden pendiente (backorder) y actualizar estados de crédito o devoluciones. Si alguna etapa del flujo falla, el sistema debe revertir todos los cambios (rollback) para evitar ventas de mercancía ya comprometida o inconsistencias de inventario.

- **Alternativas:**
    1. **Acceso Directo a BD (SQL/ORM en Servicios de Aplicación):** Ejecutar sentencias SQL o llamadas al ORM directamente en los servicios. **Consecuencia**: Alto acoplamiento entre la aplicación y el motor de base de datos, imposibilidad de probar la lógica sin una BD activa y riesgo de transacciones parciales no revertidas ante errores.

2. **Repository Pattern + Unit of Work:** Abstraer el almacenamiento mediante interfaces de repositorio en lenguaje de dominio (IVentaRepository, IInventarioRepository) y gestionar el límite transaccional con Unit of Work. **Consecuencia**: Aislamiento total del dominio respecto al almacenamiento y garantía de atomicidad en operaciones complejas.

- **Decisión:** Implementar el patrón Repository para abstraer el almacenamiento y recuperación de agregados del dominio, coordinado por Unit of Work para garantizar transacciones atómicas (commit / rollback) en operaciones que afecten a múltiples repositorios dentro de un mismo caso de uso.

- **Consecuencias:**
    1. **Beneficios**: El dominio permanece libre de dependencias de infraestructura (SQL, ORM o base de datos); se garantiza la consistencia e integridad transaccional en operaciones críticas como la reserva e inventario en tránsito.

    2. **Costos y Riesgos**: Requiere escribir interfaces e implementaciones concretas en la capa de infraestructura, incrementando la cantidad de clases y archivos en la solución.

- **Trazabilidad: BRD:** N1, N12, N13, N17; **Historias de Usuario:** US-001, US-012, US-013, US-017; **Glosario:** Mercancía comprometida, Backorder; **BPMN:** Flujo de reserva y transferencia de stock.

ADR-003: Aplicación de CQRS Básico para la Separación de Escritura (Commands) y Lectura (Queries)

- **Estado:** Aceptada

- **Contexto:** En FerreMax existe una alta frecuencia de consultas simultáneas de disponibilidad de inventario por SKU, pasillos y precios desde múltiples sucursales, junto con la necesidad de alimentar un dashboard gerencial en tiempo real. Al mismo tiempo, las operaciones de escritura (crear cotización, procesar venta, autorizar crédito o registrar transferencia) exigen validaciones de negocio e integridad estricta. Consultar datos a través de los agregados pesados del dominio ralentiza la lectura y genera cuellos de botella.

- **Alternativas:**
    1. **Modelo Único de Lectura y Escritura:** Utilizar las mismas entidades de dominio para realizar las consultas de pantalla y las modificaciones de estado. **Consecuencia:** Consultas lentas por hidratación innecesaria de agregados complejos y riesgo de bloqueos de tabla durante picos de venta en mostrador.

    2. **CQRS Básico (Commands & Queries Separados):** Separar los flujos de modificación de estado (Commands procesados por el Domain Model) de los flujos de lectura (Queries que devuelven DTOs planos optimizados). **Consecuencia:** Mayor rendimiento en lecturas de alta frecuencia y claridad de responsabilidades en la aplicación.

- **Decisión:** Aplicar CQRS básico en la capa de aplicación. Las operaciones que modifican estado (RegistrarVentaCommand, ReservarStockCommand) serán procesadas por Handlers que invocan al Domain Model. Las consultas de información (ConsultarStockSKUQuery, ObtenerDashboardGerencialQuery) devolverán DTOs de lectura (View Models) optimizados sin pasar por los agregados de escritura.

- **Consecuencias:**
    1. **Beneficios**: Desempeño optimizado en consultas masivas de inventario por SKU y renderizado del dashboard gerencial; separación explícita entre intenciones de modificación y necesidades de visualización.

    2. **Costos y Riesgos**: Duplicación de estructuras de datos (DTOs de lectura vs. clases de dominio de escritura) que requiere mantener la sincronización conceptual cuando cambien los requerimientos.

- **Trazabilidad: BRD:** N3, N19; **Historias de Usuario:** US-003, US-019, US-020; **Glosario:** Dashboard gerencial, SKU, Existencia.

Diagramas UML

Diagrama de clase

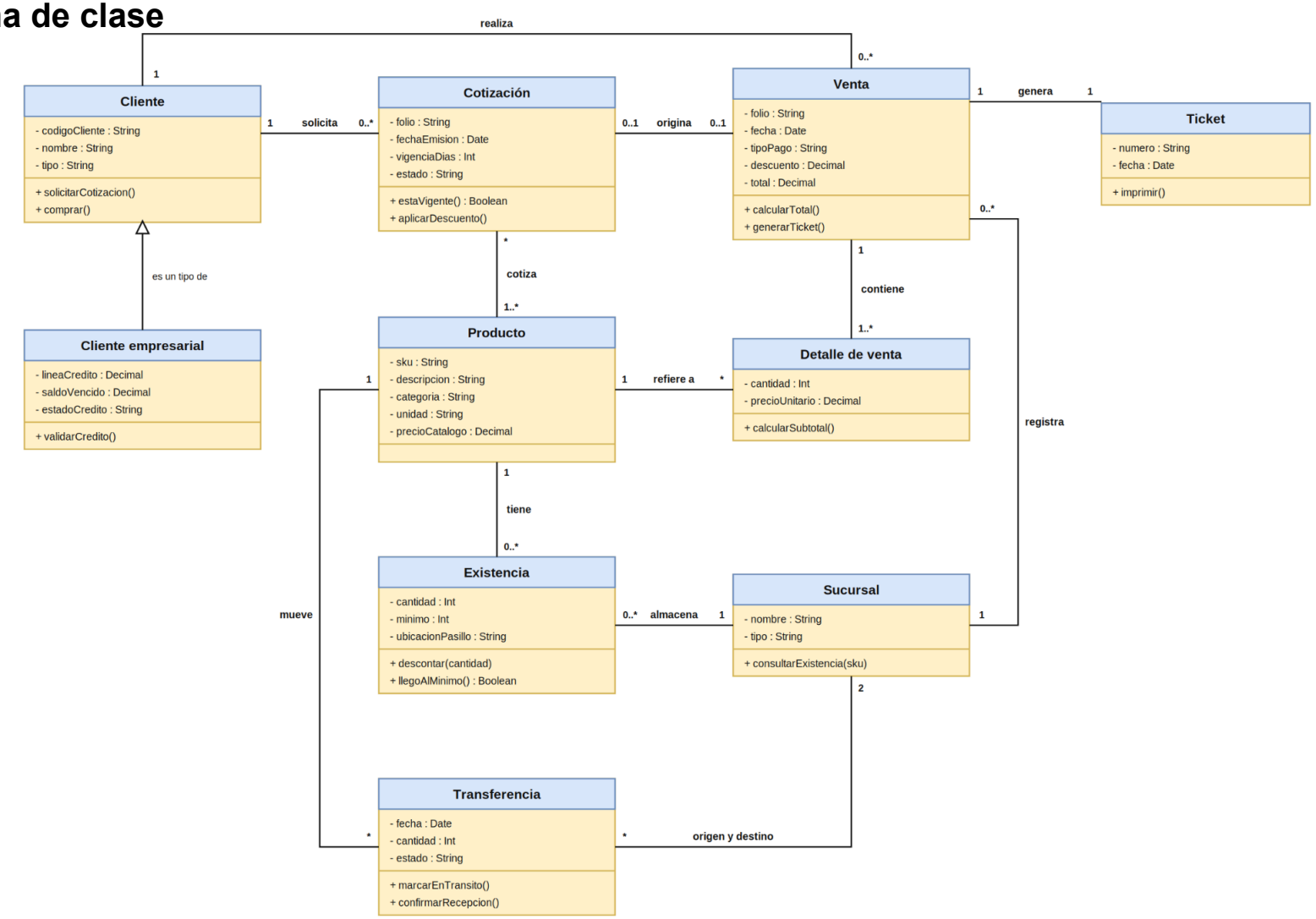


Imagen 2 Diagrama de Clases

Diagrama de Secuencia CU-01

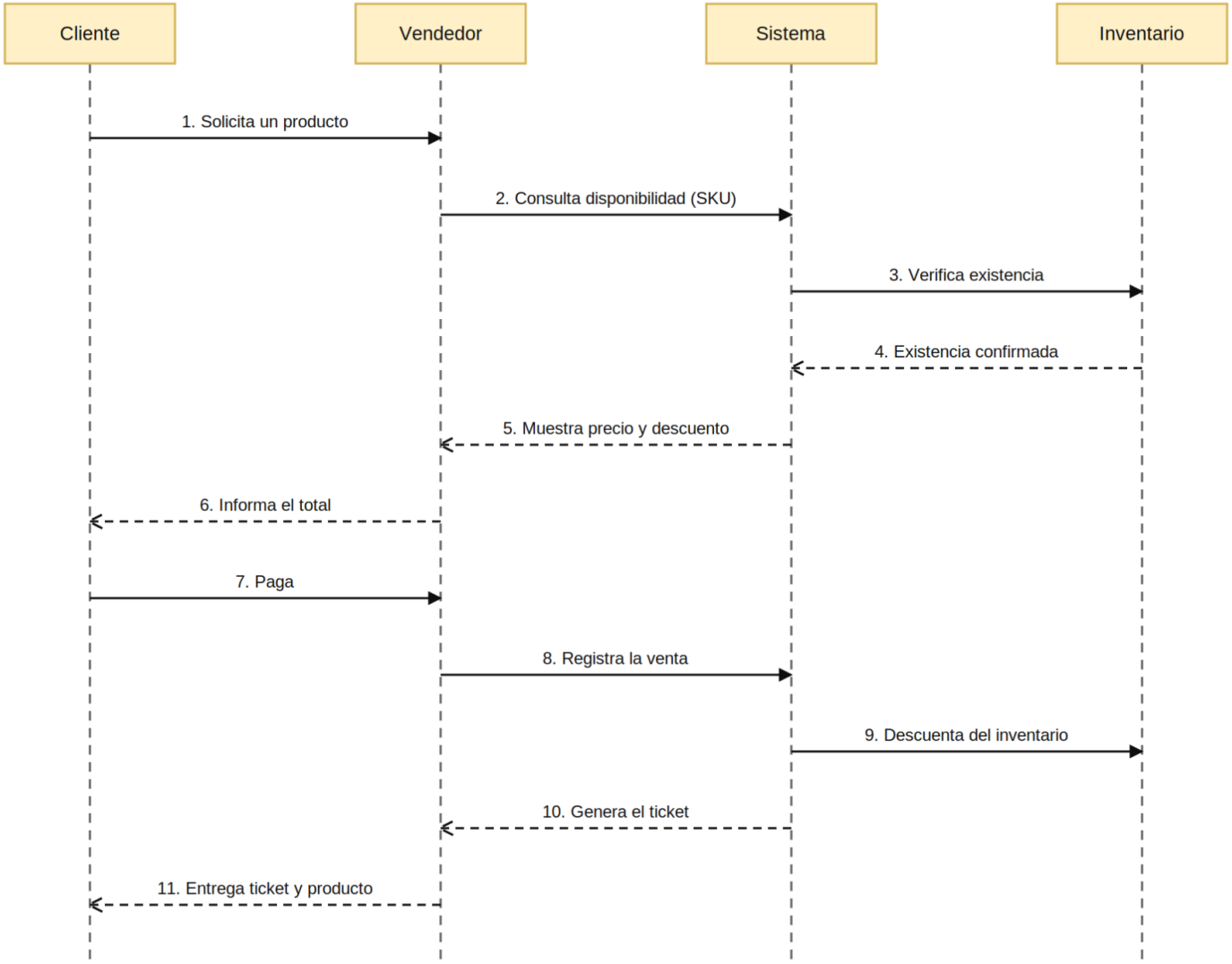


Imagen 3 Diagrama de secuencia CU-01

Diagrama de Secuencia CU-04

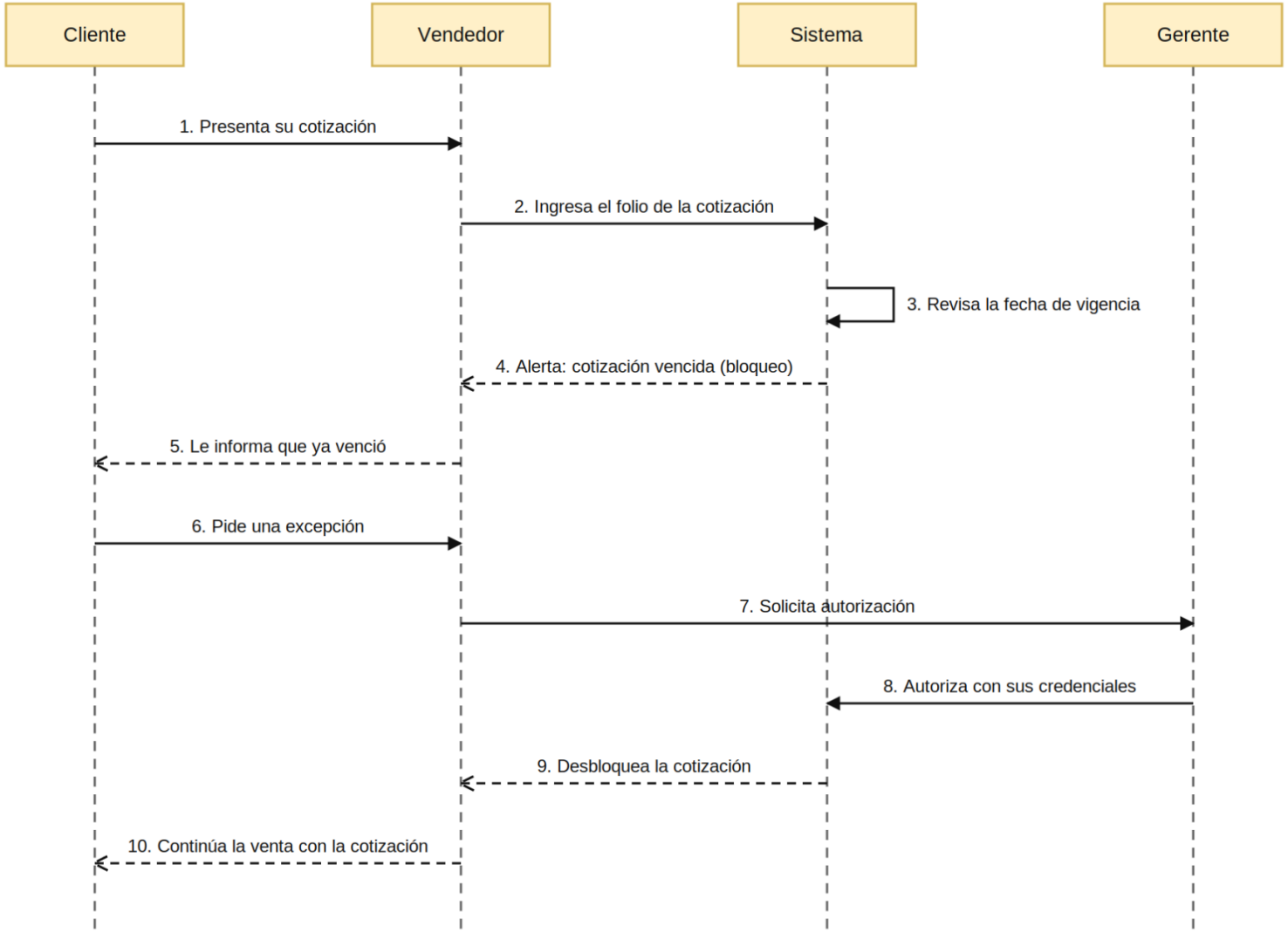


Imagen 4 Diagrama de secuencia CU-04

9. Conclusiones

El diseño arquitectural desarrollado para FerreMax transforma las necesidades del negocio identificadas en la Unidad 2 en una estructura técnica limpia, desacoplada y robusta. La separación en capas (Dominio, Aplicación e Infraestructura), la adopción del patrón Domain Model y el aislamiento de la persistencia garantizan la continuidad de la operación y protegen la rentabilidad de la empresa ante cualquier cambio en el entorno tecnológico.

Impacto Arquitectural

- **Protección de Reglas de Negocio y Erradicación de Ventas Dobles:** Mediante la reserva de stock en tiempo real gestionada por el Aggregate de `Existencia` con Control de Concurrencia Optimista (OCC) y la coordinación atómica de transacciones a través de `Unit of Work`, se eliminan definitivamente las promesas de mercancía comprometida entre sucursales.
- **Garantía de Desacoplamiento Tecnológico y Escalabilidad Futura:** La inyección de abstracciones mediante interfaces de `Repository` desvincula el núcleo operativo de las tecnologías de base de datos relacional o frameworks de UI, habilitando la operación transitoria en *Modo Offline* en sucursales. Además, esta arquitectura desacoplada permite la futura integración de nuevos canales de venta (como una plataforma de comercio electrónico B2B) o la apertura de nuevas sucursales físicas sin necesidad de modificar ni reescribir las reglas del Dominio central.
- **Inviolabilidad de Invariantes Comerciales y Crediticias:** La encapsulación de reglas en los agregados `Cotizacion` y `ClienteEmpresarial` impide la aplicación en caja de cotizaciones vencidas (requiriendo credenciales PIN de Gerente) y detiene automáticamente las ventas a clientes en estado "Crédito suspendido".

Matriz de Trazabilidad

ID Necesidad BRD	Historia de Usuario (US)	Caso de Uso (U3) / Proceso	Regla de Negocio / Glosario	Componente de Arquitectura (Dominio / App / Infra)	Decisión Arquitectural (ADR)
N1	**US-001**, **US-019**	**CU-01:** Reserva y Venta de Stock	Descuento inmediato e impedir venta de mercancía comprometida.	Agregados Venta / Pedido, interfaces IVentaRepository e IUnitOfWork	**ADR-001** (Domain Model) y **ADR-002** (Unit of Work).
N2	**US-002**	**CU-01 / CU-03:** Consulta y Transferencia	Alerta de stock 0 local y oferta de transferencia desde otra sucursal.	InventarioService (App), TransferenciaService.	**ADR-001** (Domain Model).
N3	**US-003**	Consulta de Catálogo y Ubicación	Búsqueda por SKU con existencia, stock mínimo y pasillo/anaquel.	ConsultarStockSKUQuery (CQRS), StockSKUDTO.	**ADR-003** (CQRS - Lectura Separada).

ID Necesidad BRD	Historia de Usuario (US)	Caso de Uso (U3) / Proceso	Regla de Negocio / Glosario	Componente de Arquitectura (Dominio / App / Infra)	Decisión Arquitectural (ADR)
N4	**US-004**	**CU-04:** Autorizar Excepción de Cotización	Bloqueo automático por fecha de expiración; requiere PIN de Gerente.	Agregado Cotizacion, CotizacionService, Domain Event CotizacionVencidaEvent.	**ADR-001** (Domain Model - Protege Invariante).
N5	**US-005**	Generación de Cotizaciones	Vigencia automática: 5 días regular, 3 días cliente empresarial o mayoreo.	Agregado Cotizacion, Value Objects Polimórficos PoliticaVigencia y PoliticaDescuento	**ADR-001** (Domain Model).
N6	**US-006**	Generación de Cotizaciones / Venta	Descuento automático: 5% cliente regular, 10% cliente empresarial.	Agregado Cotizacion, Value Objects Polimórficos PoliticaVigencia y PoliticaDescuento.	**ADR-001** (Domain Model).
N7	**US-007**	Validación de Crédito en Caja	Consulta automática de línea de crédito y facturas vencidas por código.	Agregado ClienteEmpresarial (exclusivo, sin métodos de crédito en clase base Cliente)	**ADR-001** (Domain Model).

ID Necesidad BRD	Historia de Usuario (US)	Caso de Uso (U3) / Proceso	Regla de Negocio / Glosario	Componente de Arquitectura (Dominio / App / Infra)	Decisión Arquitectural (ADR)
N8	**US-008**	Validación de Crédito en Caja	Mensaje y estado "Crédito suspendido" por exceso de límite o saldo vencido.	Agregado ClienteEmpresarial, Domain Event CreditoSuspendidoEvent.	**ADR-001** (Domain Model).
N9	**US-009**	**CU-04:** Autorización de Excepción de Crédito	Solicitud remota desde caja y aprobación en dashboard de Gerente.	ExcepcionCreditoService (App), IClienteRepository.	**ADR-001** (Domain Model).
N10	**US-010**	Programación de Logística	Agrupamiento automático de entregas a domicilio por Código Postal.	Agregado HojaDeRuta, RuteoService (Domain Service).	**ADR-001** (Domain Model).
N11	**US-011**	Programación de Logística	Límite de hora (6:00 PM), validación de capacidad de vehículo y zona segura.	Domain Service ValidacionLogisticaServic e.	**ADR-001** (Domain Model).

ID Necesidad BRD	Historia de Usuario (US)	Caso de Uso (U3) / Proceso	Regla de Negocio / Glosario	Componente de Arquitectura (Dominio / App / Infra)	Decisión Arquitectural (ADR)
N12	**US-012**	Procesamiento de Pedido Parcial	Cobro de stock disponible y generación automática de orden pendiente (Backorder).	Agregado Pedido, Value Object Backorder, PedidoService.	**ADR-001** y **ADR-002** (Unit of Work).
N13	**US-013**	**CU-03:** Transferencia entre Sucursales	Mercancía en trasladó pasa a estado "en tránsito" y se bloquea su venta.	Agregado Transferencia, Value Object EstadoMercancia.	**ADR-001** y **ADR-002** (Repository / UOW).
N14	**US-014**	Procesamiento de Devolución	Escaneo de ticket y antigüedad máxima de 30 días para proceder.	Agregado Devolucion, Value Object AntiguedadTicket.	**ADR-001** (Domain Model).

ID Necesidad BRD	Historia de Usuario (US)	Caso de Uso (U3) / Proceso	Regla de Negocio / Glosario	Componente de Arquitectura (Dominio / App / Infra)	Decisión Arquitectural (ADR)
N15	**US-015**	Procesamiento de Devolución	Reembolso obligatorio por el mismo método de pago original de la venta.	Agregado Devolucion, Value Object MetodoPago.	**ADR-001** (Domain Model).
N16	**US-016**	Venta Excepcional por Desajuste	Permite venta de stock "0" con justificación obligatoria, ajuste -1 y alerta a auditoría.	RegistrarVentaForzadaCommand, Domain Event VentaInconsistenteEvent.	**ADR-001** y **ADR-003** (CQRS - Command).
N17	**US-017**	Operación en Sucursal	Modo offline temporal para ventas en efectivo/crédito y sincronización posterior.	OfflineSyncService (Infraestructura), Queue local de transacciones.	**ADR-002** (Unit of Work).

ID Necesidad BRD	Historia de Usuario (US)	Caso de Uso (U3) / Proceso	Regla de Negocio / Glosario	Componente de Arquitectura (Dominio / App / Infra)	Decisión Arquitectural (ADR)
N18	**US-018**	Reabastecimiento de Inventario	Alerta automática al módulo de Compras al llegar al nivel de stock mínimo.	Domain Event StockMinimoAlcanzadoEvent, Handler NotificacionComprasHandler.	**ADR-001** (Domain Events).
N19	**US-020**	Panel de Control Gerencial	Dashboard en tiempo real con ventas diarias, top productos y cartera vencida.	ObtenerDashboardGerencialQuery, DTO DashboardViewModel.	**ADR-003** (CQRS - Lectura Separada).

Bitácora de IA (entregable 5)

Iteración	Objetivo	Prompt o enlace	Respuesta útil	Decisión del equipo
01	Comparar patrones de lógica de negocio (Transaction Script vs Domain Model) para el caso FerreMax.	“Estructura teórica comparativa basada en Martin Fowler y cuadro de ventajas/desventajas”	La IA sugirió implementar Transaction Script por ser "más simple de programar", ignorando los problemas de concurrencia en la reserva de stock y créditos suspendidos.	Rechazo de propuesta. Se seleccionó Domain Model para proteger las invariantes complejas (descuentos 5%/10%, vigencias de 3/5 días e integridad de inventario en tiempo real).
02	Generar borrador del sintáctico en PlantUML para las entidades principales del sistema.	"Genera un diagrama de clases DDD en PlantUML para FerreMax con Cliente, Cotización, Venta e Inventario."	Estructura sintáctica básica en PlantUML con relaciones de agregación y composición.	Corrección por violación de LSP. Se reestructuró la jerarquía dejando el crédito exclusivamente en el Agregado ClienteEmpresarial, evitando errores en clientes particulares.
03	Auditar el diseño de software frente a los 5 principios SOLID.	“Revisa el diagrama de clases y código para detectar violaciones a los principios SOLID (SRP, OCP, LSP, ISP, DIP) en FerreMax.”	Identificó el acoplamiento directo entre la capa de aplicación y la base de datos MySQL.	Ajuste de diseño. Se crearon Value Objects polimórficos de políticas (PoliticaDescuento, PoliticaVigencia) para permitir nuevos perfiles de cliente sin modificar código probado.
04	Redactar los borradores de los Registros de Decisiones Arquitecturales (ADRs).	“Genera los borradores para ADR-001 (Patrón de Negocio), ADR-002 (Persistencia y UOW) y ADR-003 (CQRS) siguiendo el formato estándar para FerreMax.”	Estructuró un formato de 6 campos para documentar contexto, alternativas, decisiones y consecuencias para cada uno de los ADRs	Se rechazó la propuesta de Transaction Script: Se adoptó formalmente Domain Model y se redactó la trazabilidad explícita vinculando las necesidades N1-N19 y las Historias de Usuario US-001 a US-020

05	Construir la Matriz de Trazabilidad completa.	“Crea una matriz de trazabilidad que cruce las 19 necesidades del BRD (N1-N19), las Historias de Usuario (US-001-US-020) y los componentes de arquitectura”	Estructuró en formato de tabla cruzando los requisitos con las historias de usuario y reglas de negocio.	Se estructuró en formato de TABLA matricial: Se construyó la matriz completa de 6 columnas relacionando cada necesidad del BRD con su componente DDD, caso de uso y ADR asociado
06	Organizar la estructura física del repositorio en GitHub.	Estructura un repositorio de GitHub para un proyecto de arquitectura DDD con carpetas para docs, diagrams, src y tests.	Propuso un llayout de directorio estándar separando documentación, diagramas UML y código fuente.	Se limpió la raíz moviendo archivos a subcarpetas dedicadas (docs/ADRs/, docs/soporte_unidades/, diagrams/sources/, diagrams/exports/, src/Domain, src/Application, src/Infrastructure)
07	Consolidar la Bitácora de IA y el Informe Final de Arquitectura PDF.	Consolida la bitácora de IA y el informe final en PDF integrando las etapas 1 a 8, el análisis SOLID y las conclusiones	Planifico la plantilla de organización del reporte final en PDF con secciones formales por cada etapa de la guía.	Se registraron explícitamente los errores detectados en las respuestas de la IA y las correcciones humanas reales aplicadas sobre el diseño final.

Código Base

```
src/
├── FerreMax.Domain/
│   ├── Aggregates/
│   │   ├── Cotizacion.cs
│   │   ├── Venta.cs
│   │   ├── Existencia.cs
│   │   └── Transferencia.cs
│   ├── Entities/
│   │   ├── Cliente.cs
│   │   ├── ClienteEmpresarial.cs
│   │   └── Producto.cs
│   └── ValueObjects/
│       ├── PoliticaVigencia.cs
│       └── PoliticaDescuento.cs
└── FerreMax.Infrastructure/
    ├── Interfaces/
    │   ├── IRepository.cs
    │   ├── IVentaRepository.cs
    │   ├── IInventarioRepository.cs
    │   ├── ICotizacionRepository.cs
    │   ├── IClienteRepository.cs
    │   └── IUnitOfWork.cs
    └── Persistence/
        ├── FerreMaxDbContext.cs
        └── UnitOfWork.cs
```